

Predict-Observe-Explain: Hands-On Exercises and Interactive Learning Materials for Machine Learning Courses

Ricardo Inácio; Ribeiro, Rita P.; Moisés Santos; Carlos Soares

Table of contents

1 Purpose and scope	1
2 Pedagogical design	1
2.1 Predict Observe Explain	1
2.2 Analytical and computational activities	2
2.3 Interactive experimentation	2
3 Learning progression	2
4 Use in an LLM rich learning environment	4
5 Instructor use and assessment	4
6 Technical reproducibility	5
7 Reuse and adaptation	5

Purpose and scope

This publication provides a coherent set of materials for an undergraduate introductory course in machine learning. It combines analytical exercises, reproducible computational experiments, interactive controls, supplied datasets, and parallel student and instructor editions. The materials are intended for learners who have basic Python familiarity but are still developing the conceptual vocabulary needed to reason about model assumptions, evaluation, and behaviour.

The central instructional sequence is **Predict, Observe, Explain**. Before executing an experiment, students state what they expect and why. They then run a controlled comparison, inspect its numerical or visual outcome, and explain the result in relation to the model or evaluation procedure. The sequence makes computation an instrument for testing ideas rather than an end in itself.

This guide documents the design rationale, intended use, learning progression, instructor resources, and reproducibility provisions of the collection. It describes a pedagogical design; it does not claim that the design has yet been validated through a controlled study of learning outcomes.

Pedagogical design

Predict Observe Explain

Each notebook experiment is organised as a short cycle with three distinct cognitive tasks.

Stage	Student action	Instructional purpose
Predict	Commit to an expected result before execution and provide a reason.	Activates prior knowledge and makes assumptions visible.
Observe	Run the supplied experiment, manipulate the relevant controls, and record the salient result.	Produces concrete evidence while keeping attention on a bounded concept.

Stage	Student action	Instructional purpose
Explain	Reconcile the observation with the prediction using appropriate machine-learning concepts.	Develops causal and comparative reasoning beyond reporting an output.

The prediction is not assessed only as right or wrong. Its primary value is to create a testable expectation. An incorrect prediction can be productive when the subsequent explanation identifies the assumption that failed. Conversely, a correct prediction is incomplete unless the student can connect the observed pattern to model capacity, geometry, data properties, or evaluation design.

The notebooks therefore provide the experimental apparatus and leave the intellectual work visible. Setup and helper code are collapsed where appropriate, while questions, controls, plots, metrics, and response spaces remain central. This structure limits avoidable implementation overhead without converting the activity into a passive demonstration.

Analytical and computational activities

The written handouts and notebooks address related concepts through different forms of reasoning. Pen-and-paper exercises expose quantities and transformations that software often hides: impurity calculations, distance measures, forward and backward passes, regression residuals, cross-validation summaries, clustering updates, meta-features, and association-rule measures. Worked solutions make each intermediate step inspectable.

The computational activities then place those concepts in less artificial settings. Students vary model capacity, kernels, neighbourhoods, thresholds, feature scaling, initialisation, and clustering assumptions; compare estimates across shared data splits or folds; and inspect decision boundaries, fitted surfaces, score distributions, residuals, rankings, or recommendation outcomes. The analytical task establishes how a method works, while the computational task reveals how that mechanism behaves when data and configurations change.

The two modes need not be used in a fixed order. An instructor may use a calculation before a notebook experiment to establish a mechanism, or use the experiment first to generate a phenomenon that the calculation subsequently explains. In either case, students should be asked to connect the representations rather than treat the handout and notebook as independent assignments.

Interactive experimentation

Interactive cells support controlled counterfactual questions such as what changes when a tree becomes deeper, an RBF kernel becomes more local, a neighbourhood grows, a decision threshold falls, or a clustering parameter changes. The instructional value lies in comparing configurations while holding the dataset and surrounding procedure fixed. Students should be encouraged to vary one consequential control at a time, describe both stable and changing features of the output, and distinguish training behaviour from evidence about generalisation.

The controls are not intended as an unguided search for the best score. They are used to expose sensitivity, inductive bias, trade-offs, and failure modes. A useful discussion therefore asks not only which setting performed better, but also what changed in the learned structure, which observations were affected, whether the result is likely to persist, and what additional evidence would be needed.

Learning progression

The collection moves from data inspection and local model behaviour to empirical comparison across algorithms and datasets. The sequence is cumulative but modular: individual units can be adopted independently when their prerequisites are met.

Unit	Main focus	Intended learning outcomes
HO01	Environment and data exploration	Inspect tabular data; distinguish feature types from storage types; filter, group, and compare distributions; recognise how class frequency can affect summaries.
HO02	Decision trees and evaluation	Interpret axis-aligned splits and tree decisions; inspect classification errors; relate depth and leaf constraints to capacity; recognise variability in cross-validation estimates.
HO03	Neighbourhoods and probabilistic classification	Reason about distance and scaling; relate neighbourhood size and weighting to boundary smoothness; compare local, probabilistic, and tree-based inductive biases.
HO04	Support vector machines and neural networks	Compare linear and nonlinear decision geometry; reason about kernel and regularisation controls; relate network capacity and initialisation to learned boundaries; calculate elementary neural-network updates.
HO05	Linear and tree regression	Interpret predictions, residuals, MAE, and MSE; compare linear and piecewise-constant surfaces; distinguish training fit from test performance.
HO06	Empirical algorithm comparison	Use shared folds for fair comparison; interpret mean performance and variability; compare dataset-level ranks; understand what the Friedman test does and does not establish.
HO07	Clustering	Contrast centroid, density, and hierarchical assumptions; explain the effect of scale and distance; interpret inertia, silhouette values, noise assignments, and dendrogram merges.
HO08	Ensembles and imbalanced classification	Calculate majority votes and classification metrics; identify misleading accuracy under imbalance; analyse the effects of class weighting and threshold choice.
HO09	Meta-learning and algorithm selection	Treat datasets as observations; construct and interpret meta-features; relate task characteristics to performance; predict and rank candidate algorithms for unseen datasets.

Unit	Main focus	Intended learning outcomes
HO10	Association rules	Calculate support, confidence, and lift; construct frequent itemsets and rules; explain why confidence alone can be misleading.

HO01 introduces the working environment and empirical habits through a notebook. HO02 to HO09 pair computational notebooks with written handouts and worked solutions. HO10 concludes the collection with written exercises and solutions on association-rule learning. This variation reflects the concepts and activities currently implemented rather than an assumption that every topic requires the same medium.

Use in an LLM rich learning environment

Generative systems can produce plausible code and explanations quickly, so syntax production alone provides increasingly weak evidence of understanding. These materials shift the assessable work towards prior prediction, interpretation of evidence, comparison of controlled alternatives, and explanation of unexpected outcomes. A student may use an LLM in accordance with the course policy, but still needs to make and defend claims that refer to the actual experiment.

Instructors can preserve this emphasis by assessing the reasoning trace rather than the mere presence of executable code. Strong responses identify the expected direction of change, cite a specific observation, connect it to a mechanism, and acknowledge uncertainty or alternative explanations. Weak responses paraphrase a plot, report a score without comparison, or offer a generic model description that could have been written before the experiment.

Where LLM use is permitted, students can be required to verify generated claims against the notebook output and to identify at least one limitation of the proposed explanation. Short oral follow-ups, parameter choices justified before execution, and comparisons using instructor-selected configurations can further establish that the student understands the result. The objective is not to make assistance undetectable or unusable; it is to design work for which unexamined assistance is insufficient.

Instructor use and assessment

The student materials contain prompts and response spaces without completed answers or stored outputs. Matching solution notebooks retain the same experimental structure and code while adding concise interpretations and reproducible outputs. Written solution sheets expose the calculations and reasoning expected for the corresponding handouts. Instructor editions should be distributed separately when the activities contribute to assessment.

A typical session can use the following structure:

1. Introduce the concept and elicit an initial qualitative expectation.
2. Complete or discuss a short analytical exercise that exposes the relevant mechanism.
3. Ask students to record a prediction for a bounded notebook experiment.
4. Run the experiment and vary one meaningful control.
5. Compare observations in pairs or small groups before writing an explanation.
6. Conclude by identifying what the experiment supports, what it does not establish, and how the result may change on another dataset.

Assessment can separate the three stages instead of assigning a single mark to a completed notebook. Predictions can be evaluated for specificity and justification, observations for faithful use of evidence, and explanations for conceptual accuracy and reconciliation with the prediction. This approach rewards revision of reasoning rather than hindsight. For larger assignments, students can design a small extension that changes one assumption, state a falsifiable expectation, and explain whether the new evidence supports it.

The solution materials are examples of adequate reasoning, not unique model answers. Alternative explanations should be accepted when they are consistent with the evidence and the scope of the experiment. Instructors

should also distinguish random variation from a stable behavioural pattern and avoid treating a result from one split or dataset as a universal property of an algorithm.

Technical reproducibility

The repository includes the datasets used by the activities, provenance and integrity information where applicable, a reproducible Python environment, shared course utilities, Quarto sources, and pre-rendered PDF and Word documents. Student and solution notebooks are structurally paired. Automated checks validate notebook structure, dataset integrity, referenced media, and reproducibility requirements. The full publication workflow executes the notebooks before regenerating the documents.

For ordinary use, learners can create the supplied environment, install the local package, and open the notebooks in JupyterLab or a compatible editor. Pre-rendered handouts require no build tools. Instructors who modify the written materials can regenerate the PDF and Word editions with Quarto; a complete pre-publication build additionally runs validation, tests, and notebook execution.

Reproducibility here means that the released materials, data, environment specifications, and documented commands are sufficient to repeat the supplied activities and regenerate the published documents under the supported setup. It does not imply bit-for-bit identity across every operating system or library update. Versioned releases should therefore preserve the exact teaching materials used in a course edition, and substantive revisions should receive a new release version.

Reuse and adaptation

The units may be taught as a complete sequence or selected individually. When adapting an activity, instructors should preserve the alignment between its prediction prompt, manipulated variable, observable evidence, and explanation question. Adding controls without a corresponding question can turn a bounded investigation into parameter browsing; removing the prediction stage can make the explanation retrospective and less diagnostic of prior understanding.

Teaching content is licensed under CC BY 4.0 and code under the MIT License. Third-party datasets retain their stated terms. Reusers should preserve attribution, dataset notices, and the distinction between student and instructor materials. The repository README contains the current setup, build, licensing, and citation information for the released version.